# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME:** INTERNET PROGRAMMING
**COURSE CODE:** CSA43

**NAME:** B KIRTHANAA
**REGISTER NUMBER:** 192324244

---

### COURSE OUTCOME COVERED

**CO-1 :** Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

---

**QUESTIONS:** **Total Marks: 30**

## Question 1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

**Questions:**

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

## SOLUTION:

### 1. Identify appropriate HTML5 semantic elements missing in the structure

HTML5 semantic elements give a webpage meaningful structure and make it easier for browsers, developers, and screen readers to understand the content. The given snippet is only a bare form; it lacks the document skeleton, the page-level semantic elements, and labels for its inputs.

**Document structure elements missing:**

- **<!DOCTYPE html>** – Declares the document as HTML5.
- **<html>** – Defines the root of the HTML document.
- **<head>** and **<title>** – Contain metadata and the page title.
- **<body>** – Contains the visible webpage content.

**Semantic elements missing:**

- **<header>** – Contains the heading of the symposium portal.
- **<main>** – Contains the main registration content.
- **<section>** – Groups the related registration form content.
- **<footer>** – Contains copyright or contact information.
- **<label>** – Associates a visible caption with each input field.

### 2. Modify the structure to make it standards-compliant

The following code uses HTML5 semantic elements, labels, and appropriate input attributes.

**HTML CODE:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Symposium Registration</title>
</head>
<body>

  <header>
    <h1>Online Symposium Portal</h1>
  </header>

  <main>
    <section>
      <h2>Student Registration</h2>

      <form action="/register" method="POST">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>

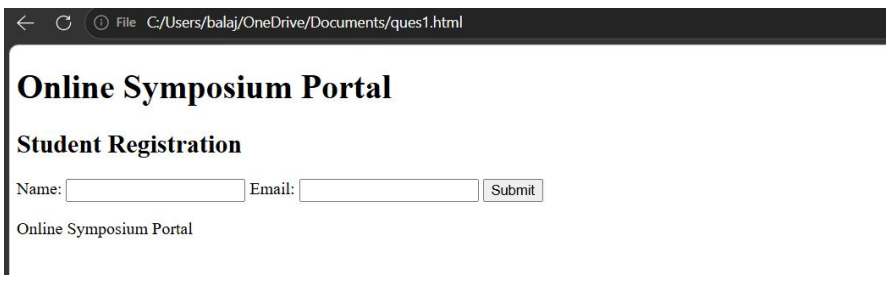
        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>

        <button type="submit">Submit</button>
      </form>
    </section>
  </main>

  <footer>
    <p>Online Symposium Portal</p>
  </footer>

</body>
</html>
```

**OUTPUT:**



### 3. Interpret the HTTP request generated during form submission

When a student fills in the name and email fields and clicks the Submit button, the browser generates an HTTP POST request and sends it to the server endpoint /register. The submitted information is included in the request body.

**HTTP REQUEST:**

```
POST /register HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 44

name=Kirthanaa&email=kirthanaa%40example.com
```

The request contains the following information:

- **POST:** Specifies the HTTP method used to submit the form.
- **URL:** /register identifies the server resource receiving the data.
- **Content-Type:** Indicates that the data is encoded as form data.
- **Request body:** Contains the student's name and email address.

Unlike a GET request, the POST request sends the form data in the request body rather than appending it to the URL. However, HTTPS is required to protect the data during transmission.

### 4. Analyze the HTTP response cycle from server to browser

The HTTP response cycle describes how the server processes a registration request and sends the result back to the browser. After receiving the POST request, the server validates the submitted information and, if the application supports it, stores the registration details in a database.

**The process involves these steps:**

1. **Form submission:** The student enters the details and clicks Submit.
2. **Request transmission:** The browser sends the HTTP POST request to the server.
3. **Server processing:** The server validates the information and processes the registration.
4. **Response generation:** The server returns an HTTP response containing a status code and, where appropriate, a confirmation message.
5. **Browser display:** The browser displays the response to the student.

**Common HTTP response status codes include:**

- **200 OK** – The request was processed successfully.
- **201 Created** – A new registration resource was created.
- **400 Bad Request** – The request contains invalid or incorrect data.
- **500 Internal Server Error** – An unexpected server-side error occurred.

Thus, the HTTP request-response cycle enables communication between the browser and the server.

### 5. Suggest improvements for usability and accessibility

The registration portal can be improved by making the form simple, accessible, secure, and easy to use. Clear instructions and appropriate validation help students complete the registration process without errors.

**Recommended improvements include:**

- **Form validation:** Use required fields and email validation to prevent incomplete or incorrectly formatted submissions.
- **Accessibility:** Provide labels for all input fields and ensure that the form supports keyboard navigation and screen readers.

- **Error handling:** Display clear messages when information is missing or invalid.
- **Responsive design:** Use CSS to ensure that the portal works properly on desktops, tablets, and mobile phones.
- **Security:** Use HTTPS and validate submitted data on the server to protect student information.
- **User feedback:** Display a confirmation message after successful registration.
- **Readable interface:** Use suitable font sizes, clear buttons, and sufficient colour contrast.

These improvements make the portal more reliable and user-friendly while supporting different users and devices.

## Question 2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
<head>
  <title>Company</title>
</head>
<body>
  <div>
    <h1>Welcome
    <p>About us
  </div>
</body>
</html>
```

**Questions:**

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

### SOLUTION:

#### 1. Identify improper nesting and missing closing tags

The given HTML code contains several structural errors that may affect how the webpage is displayed. The <h1> heading and  paragraph are missing their closing tags, and the document does not include a DOCTYPE declaration or language attribute.

**The following issues are identified:**

- **Missing DOCTYPE:** <!DOCTYPE html> is required to declare an HTML5 document and help browsers use standards mode.
- **Missing closing tags:** The </h1> and  tags are missing.
- **Incomplete document structure:** The <html> element does not specify the page language.
- **Missing semantic elements:** The content uses a generic  instead of meaningful elements such as <header> and <section>.

#### 2. Analyze how different browsers may interpret this structure

Different browsers may handle the malformed HTML differently because browsers use HTML parsing rules to recover from errors. Some browsers may automatically insert missing closing tags, while others may construct the document structure differently during error recovery.

This can lead to differences in spacing, heading appearance, paragraph layout, and the positioning of content. Without a DOCTYPE declaration, a browser may also enter quirks mode, which can cause differences in CSS layout and rendering. Therefore, writing valid HTML helps ensure consistent behaviour across browsers.

### 3. Identify missing semantic elements

Semantic HTML elements describe the purpose of different parts of a webpage and improve accessibility, readability, and maintainability.

**The appropriate elements include:**

- **<header>** – Contains the company name or welcome heading.
- **<main>** – Contains the main content of the webpage.
- **<section>** – Groups related content, such as the About Us section.
- **<h1>** – Defines the main heading of the page.
- **<p>** – Displays paragraph text.
- **<footer>** – Contains copyright or contact information.

Using these elements instead of generic containers makes the page structure easier for browsers, developers, and assistive technologies to understand.

### 4. Provide a corrected W3C-compliant HTML structure

The following code corrects the missing tags, adds HTML5 semantic elements, and follows standard HTML document structure.

**HTML CODE:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>

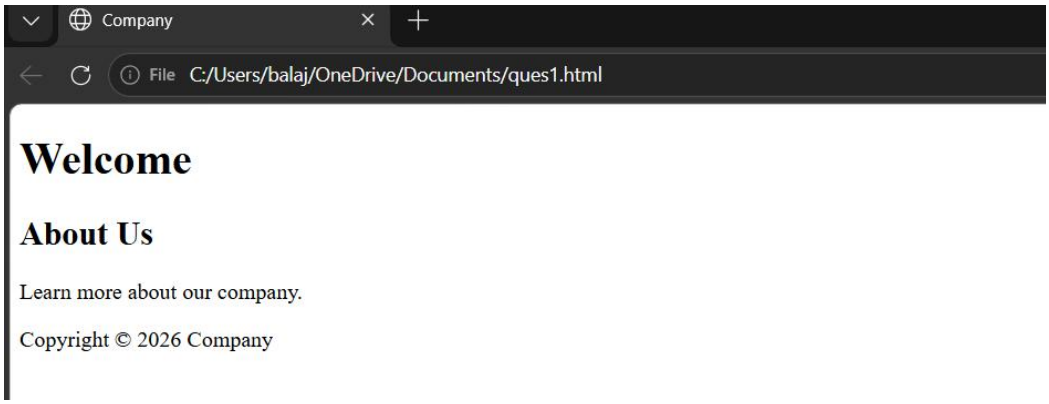
  <header>
    <h1>Welcome</h1>
  </header>

  <main>
    <section>
      <h2>About Us</h2>
      <p>Learn more about our company.</p>
    </section>
  </main>

  <footer>
    <p>Copyright © 2026 Company</p>
  </footer>

</body>
</html>
```

**OUTPUT:**




**Explanation:**

- **<!DOCTYPE html>** enables standards mode in modern browsers.
- **lang="en"** specifies the language of the document.
- The character encoding and viewport settings support correct text rendering and responsive layouts.
- All elements are properly opened and closed, with appropriate nesting.
- Semantic elements provide a clear and meaningful page structure.

### 5. Suggest techniques to ensure cross-browser compatibility

Cross-browser compatibility ensures that a website works consistently in browsers such as Chrome, Firefox, Edge, and Safari. The following techniques help achieve this:

1. **Use valid HTML:** Follow HTML5 standards and validate the code using the W3C Markup Validation Service.
2. **Use standard CSS:** Avoid relying on browser-specific behaviour and use consistent styling rules.
3. **Apply CSS resets:** Use a CSS reset or normalization stylesheet to reduce differences in default browser styles.
4. **Test across browsers:** Check the website in different browsers and on different devices.
5. **Use responsive design:** Apply flexible layouts, media queries, and appropriate viewport settings.
6. **Avoid unsupported features:** Check browser compatibility before using newer HTML, CSS, or JavaScript features.
7. **Use developer tools:** Inspect layout errors, console warnings, and rendering problems using browser developer tools.

These techniques help improve consistency, accessibility, and reliability across browsers.

## Question 3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

**Questions:**

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.

4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

## SOLUTION:

### 1. Identify issues in the form configuration

The given HTML form contains errors that prevent it from submitting data correctly. The main issue is that the Login button uses type="button" instead of type="submit". A normal button does not automatically submit the form unless JavaScript is added.

**The following issues are identified:**

- **Incorrect button type:** type="button" does not trigger form submission by default.
- **GET method:** The form uses GET, which places submitted values in the URL and is unsuitable for passwords.
- **Missing validation:** The input fields do not have the required attribute.
- **Missing labels:** The inputs have no <label>, which hurts accessibility.
- **Security issue:** Sending passwords through GET can expose them in browser history, URLs, and server logs.

### 2. Analyze why the HTTP request is not triggered

When the user enters a username and password and clicks Login, the browser does not submit the form because the button is defined as type="button". This button performs no form-submission action by default.

The browser sends an HTTP request only when the form is submitted, such as by clicking a submit button or submitting through JavaScript. Therefore, changing the button type to submit allows the browser to send the form data to the server, provided no other validation or script prevents submission.

### 3. Interpret the role of the GET method in this scenario

The GET method is used to request data from a server. When a form uses method="GET", the browser appends the submitted form values to the URL as query parameters.

**For example:**

```
/submit?username=student&password=12345
```

**The GET method has the following characteristics:**

- Data is appended to the URL rather than sent in the request body.
- URLs can be stored in browser history and server logs.
- GET requests are generally suitable for searching or retrieving information.
- GET should not be used to submit passwords or other sensitive information.

In this example, the GET method is not the main reason submission fails; the incorrect button type is the immediate cause.

### 4. Propose a corrected implementation

The form can be corrected by using method="POST", changing the button to type="submit", and adding labels and required-field validation. POST sends the form data in the request body rather than placing it in the URL.

**HTML CODE:**

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login Form</title>
</head>
<body>

  <h1>Login</h1>

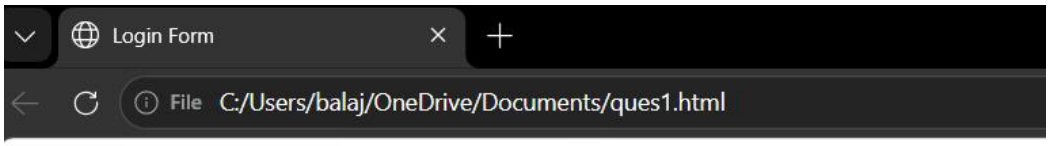
  <form action="/submit" method="POST">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required>

    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required>

    <button type="submit">Login</button>
  </form>

</body>
</html>
```

**OUTPUT:**




**Explanation:**

- **action="/submit"** specifies the server endpoint.
- **method="POST"** sends the form data in the request body.
- **type="submit"** triggers form submission.
- **required** ensures that both fields are filled in before submission.
- The labels improve accessibility and make the fields easier to identify.

The server must also be configured to accept POST requests at /submit and process the submitted credentials.

### 5. Suggest improvements for secure data transmission

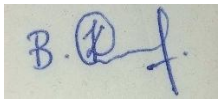
Secure data transmission is essential because login forms handle sensitive user credentials. The following improvements can help protect user information:

1. **Use HTTPS:** Encrypts data transmitted between the browser and server.
2. **Use POST for credentials:** Avoids placing passwords directly in the URL, although POST alone does not encrypt data.
3. **Validate on the server:** Verify the username and password on the server instead of relying only on browser validation.
4. **Hash passwords securely:** Store passwords using a suitable password-hashing algorithm, such as Argon2id or bcrypt, rather than storing plain-text passwords.

5. **Use secure sessions:** Protect authentication cookies with Secure, HttpOnly, and appropriate SameSite attributes.
6. **Prevent brute-force attacks:** Apply rate limiting and suitable login-attempt controls.
7. **Display safe error messages:** Avoid revealing whether a particular username exists or exposing sensitive server details.

***Code of Conduct:***

I **B KIRTHANAA 192324244** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



***Signature of the student:*** **B KIRTHANAA**

## RUBRICS FOR CALCULATION:

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Concepts | 25% | Demonstrates thorough, accurate understanding of concepts | Good understanding with minor gaps | Basic understanding with some misconceptions | Poor understanding; major misconceptions |
| Explanation | 25% | Detailed, well-explained answers with relevant examples | Clear explanation with adequate details | Limited explanation; lacks depth | Poor or unclear explanation |
| Application | 20% | Effectively applies concepts with strong analytical ability | Applies concepts with minor errors | Limited application with weak analysis | No application or incorrect analysis |
| Organization | 15% | Well-structured answers with logical flow and coherence | Organized with minor issues in flow | Basic structure, lacks clarity | Poor organization, incoherent answers |
| Accuracy | 10% | Completely accurate and covers all required points | Mostly accurate with minor omissions | Partially correct with missing points | Incorrect answers with major gaps |
| Clarity | 5% | Clear, neat, professional presentation | Understandable with minor issues | Limited clarity, readability issues | Poorly written, difficult to understand |